Assignment 2 - RAG

Due date: 28.4.26

Intro

In this assignment you'll familiarize yourself with RAG - Retrieval Augmented Generation, first in theory (just a bit) and then we'll dive into practicalities. You'll start querying without RAG and after adding the RAG component you'll compare the outcomes. The last steps will focus on RAG evaluation metrics and improvement cycles.

By the end of this assignment you'll have built a working RAG pipeline, evaluated it across three different dimensions (not just final-answer correctness), and run improvement cycles on it. The goal isn't to hit a particular accuracy number - financebench is hard on purpose - but to develop intuition for *which component* to fix when results disappoint, which is the skill that matters in production.

General Instructions

1. Submit **one** notebook that contains the code and textual answers for all tasks.
2. Some tasks also require xlsx files - **pay attention**.
3. Zip it all together and include first and last names of **both students** in the file name.
4. **Note:** I usually don't run the notebook, but there should be placeholders for the data loading, api_key etc. in case I do need to run it (it happens. Rarely, but it happens).
5. Having said that, **you** need to run it of course, and outputs should be present.

Dataset

We'll use the financebench dataset. Read the description and read carefully through the columns and make sure you understand them (you can ignore `dataset_subset_label`). The paper can also help you with that.

Notes:

1. The dataset has 3 types of questions: *metrics-generated*, *domain-relevant*, *novel-generated*. **Drop** the *metrics-generated* questions.
2. Some of the urls in the *doc_link* column are dead. Replace them with links to this repo. Note that the folder contains more documents than are referenced by the dataset.

Task 1 - naive generation (10 points)

Use the Llama-3.3-70B-Instruct model (via Nebius Token Factory) to answer the first 5 questions (simply sort by *financebench_id*) of each *question_type* - 5 domain-relevant, 5 novel-generated.

Note: No retrieval, just the question straight into the model.

For each question, compare the model's output with the expected answer (aka "ground truth" or "gold answer").

Note: the model might not provide an answer and instead ask for more information, refuse, or hedge. That's itself a result worth recording.

Look at the answers and identify:

1. Cases where the model **refused** or asked for more information - why?
2. Cases where the model **answered confidently** - spot-check against the ground truth. Is the answer correct? Partially correct? Totally wrong (hallucinated)?
3. Are there patterns by *question_type*? Do some types fail more than others?

Deliverables:

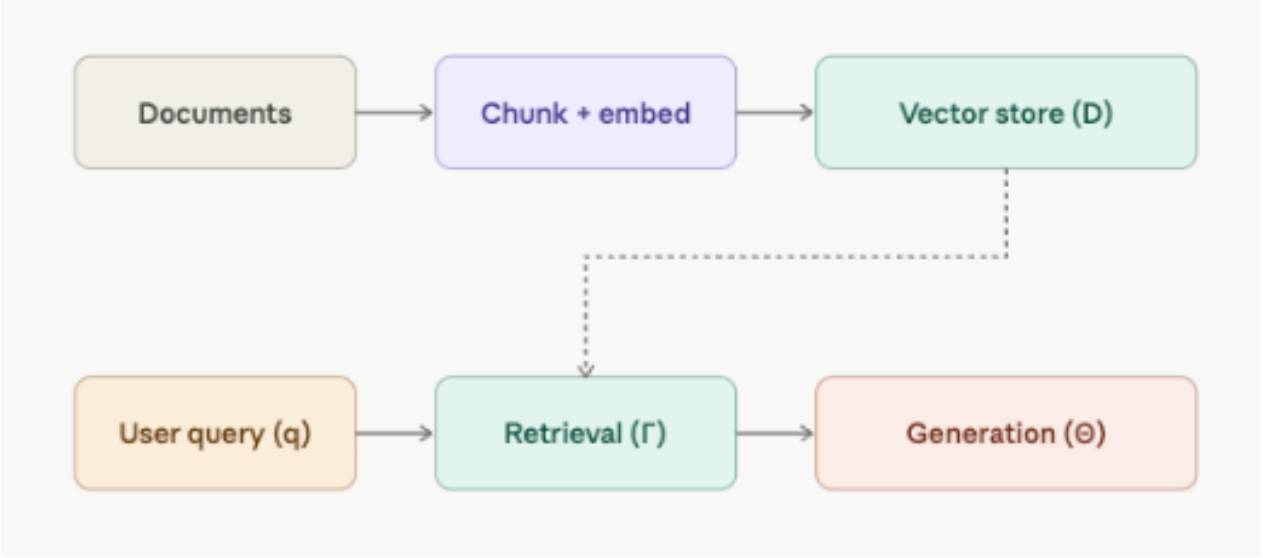
1. The code.
2. A table (xlsx) with columns: financebench_id | question_type | question | naive_answer | ground_truth | verdict, where verdict is one of {correct, partially correct, wrong, refused} based on your manual judgment.
   Name the file **assignment2_naive_generation**.
3. A short written discussion (markdown cell in the notebook) addressing the three questions above.

Task 2 - RAG reminder (5 points)

Below is a sketch of a simple RAG pipeline. The boxes group into three components - **indexing** (documents → chunk + embed → vector store), **retrieval** (user query → retrieval), and **generation**. For each component, briefly explain:

1. How does it contribute to the pipeline?
2. Where can it fail? Try to think of concrete examples.
3. Does it happen once ("offline"), or per query?

Deliverable: A short write-up (markdown cell in the notebook), ~2-4 sentences per component covering the three questions above.



Task 3 - embed documents (15 points)

The source documents (linked again here) are going to be the knowledge base for your vector store.

Instructions:

1. Load PDFs with PyPDFLoader (one Document per page). Recall that not all pdfs are relevant, use only the PDFs corresponding to *doc_name* values that appear in your filtered dataset.
2. Attach **metadata** to each page before splitting: *doc_name*, *company*, *doc_period*, *page_number*. Keep *page_number* 0-indexed to match the dataset's *evidence_page_num*.

   Note: PyPDFLoader already attaches a default page metadata field, but we're standardizing on page_number for clarity.
3. Split with RecursiveCharacterTextSplitter, chunk_size=1000, chunk_overlap=150. Chunks inherit page metadata.
4. Store in a LangChain FAISS vector store using BAAI/bge-small-en-v1.5 from Hugging-Face as the embedding model.
5. Save the FAISS index to disk (vectorstore.save_local(...)) so you don't re-embed every time you restart your notebook. You'll reuse this index across Tasks 4–7.

Pick 2-3 questions from the dataset and retrieve the top-k chunks for each. For each retrieval, check:

- Did the retrieved chunks come from the **right document** (matching *doc_name*)?
- Do they contain (or come close to) the evidence text from the dataset's *evidence* field?
- Do they come from the **right page** (chunk *page_number* vs *evidence_page_num*)?

Hint: look again at these results when working on task 7.

Deliverables:

1. The code (loading, metadata, splitting, embedding, saving).
2. A short markdown cell with your observations.

Task 4 - time for some RAG! let's build a RAG pipeline (25 points)

Write code that takes a user query, retrieves the most relevant chunks from the vector store, and feeds them - along with the query - into the generation model to produce a final answer. Use the same embedding model from Task 3 (BAAI/bge-small-en-v1.5) to embed the user's query, and the same generation model from Task 1(Llama-3.3-70B-Instruct) to produce the final answer.

Prompt construction. Think about how you format the retrieved chunks in the prompt: use clear separators between chunks, and include the *doc_name* metadata so the model knows which filing each chunk came from. Handle the case where retrieval returns an empty retrieval - the model should be told there's no relevant context rather than being handed an empty block.

System prompt. Write a system prompt that instructs the model to:

- Answer *only* from the provided context.
- Say explicitly when the context doesn't contain the answer, rather than guessing.
- Keep answers concise and cite the document each fact came from.

Deliverables:

1. The code. Make sure to include the prompts.
2. Wrap everything in a single function:
   answer_with_rag(query: str, k: int = 4) -> dict
   The returned dict should contain:
   - `answer` (str): the generation model's final answer.
   - `retrieved_chunks` (list): the chunks used as context, each with its *doc_name* and *page_number* metadata.

Task 5 - run and compare (10 points)

Run the same 10 questions from Task 1 through your RAG pipeline. For each question, put the naive answer and the RAG answer side by side with the ground truth, and comment on:

1. **Did RAG help?** Cases where the naive model refused or hallucinated, and RAG produced a grounded answer.
2. **Did RAG hurt?** Cases where the naive model happened to be right (from memorization) and RAG made it worse - e.g., retrieved the wrong filing, or pulled chunks that confused the model.
3. **Patterns by *question_type*** - does RAG help more on *domain-relevant* than on *novel-generated*, or vice versa? Any hypothesis why?

Deliverables:

1. The code.
2. An xlsx file with columns: financebench_id | question_type | question | naive_answer | RAG_answer | ground_truth
   Name the file **assignment2_run_and_compare**.
3. A discussion in a markdown cell in the notebook addressing the three points above.

Task 6 - evaluation (20 points)

The most straightforward way to evaluate a RAG pipeline is by checking the final answer's correctness - compare it with the "ground truth", as we did in the previous assignment. But we can also evaluate specific components, which enables better fine-tuning of the pipeline. Here you can find a partial list.

In this task we'll evaluate with 3 measures:

1. **Correctness** - direct comparison of the final answer and the ground truth, using an LLM as a judge. For the generation we used Llama-3.3-70B-Instruct. For the judge we'll use a different model - DeepSeek-V3-0324. Write a short judge prompt that returns a binary verdict (correct / incorrect) plus a one-sentence justification.
2. **Faithfulness** - check if the generation model actually relies on the retrieved information. Use Ragas Faithfulness from the collections API. Use *.score()* (the synchronous method), not *.ascore()*.

   Notes:

   a. For the Ragas LLM use the same model as the Correctness judge (DeepSeek-V3-0324). Wrap it using *ragas.llms.llm_factory* with an *AsyncOpenAI* client configured for Nebius Token Factory.

   b. Ragas faithfulness makes multiple LLM calls per example → pretty slow. Therefore, evaluate with this measure only for the first 20 examples in your dataset, sorted by *financebench_id*.
3. **Retrieval hit-rate** - check whether retrieval surfaced the evidence page for each question. The dataset's *evidence_page_num* field (inside the "evidence" column) tells you which page supports each answer. Use this as ground truth: for each question, retrieve the top-k chunks and check whether any of them came from the right page (compare each chunk's *page_number* metadata to *evidence_page_num*).

   Report **page-hit@k**: 1 if any retrieved chunk is from the correct page, 0 otherwise. Average across the dataset, and try a few values of k (e.g., 1, 3, 5). Note: some questions have multiple evidence items spanning different pages - count it as a hit if any evidence page is retrieved.

Instructions:

1. Run all questions in the dataset through your RAG pipeline.
2. Evaluate each answer with the above measures.

Deliverables:

1. The code.
2. A per-question xlsx file with columns: financebench_id | question | correctness | faithfulness| page_hit_at_k (column per each tested k).
   Name the file **assignment2_evaluation**.
3. Aggregate numbers: average correctness, average faithfulness, and page-hit@k for k $\in$ {1, 3, 5}.

Note: as stated in the paper, this is a hard dataset, so results will probably not be the best you've seen. Don't take it personally.

Task 7 - improvement cycles (15 points)

Run improvement experiments. For each experiment you'll run through the following steps:

1. **Hypothesis** - which metric do you expect to improve, and why? (e.g., "increasing k should improve faithfulness because the model has more relevant context to ground in, but may hurt correctness if irrelevant chunks distract it.").
2. **Change** - vary **one thing at a time** from the baseline (Task 6 results).
3. **Measure** - re-run all three metrics.
4. **Interpret** - did the metric you expected to move actually move? Did anything else move unexpectedly?

Note: for faithfulness, use the same subsample of 20 questions across all experiments. Correctness and page-hit@k should still be computed on the full dataset.

Things worth varying (pick 3 or more):

1. The generation prompt.
2. The generation model.
3. K value in top-k retrieval.
4. Chunk size.
   Note: changing chunk size means re-embedding and rebuilding the FAISS index. Save each version under a different name (e.g., faiss_chunk500, faiss_chunk1500) so you can compare.
5. Adding a reranker - use BAAI/bge-reranker-base from Hugging Face as a second pass - retrieve top-20 with FAISS, then rerank to top-4 with the cross-encoder. When varying k, vary the k that reaches the generator; keep everything else at baseline.

Deliverables:

1. A short **hypothesis** per experiment (1-2 sentences).
2. The code.
3. The **results table (xlsx)**: Task 6 baseline as the first row, followed by your experiments. Columns: experiment | change | correctness | faithfulness | page_hit_at_k (column per each tested k).
   Name the file **assignment2_improvement_cycles**.
4. A short **interpretation** per experiment (1-2 sentences).
5. A **wrap-up** (a short paragraph) answering: where does your pipeline fail most - retrieval, generation, or both? If you had one more week, what would you try next?

NEBIUS ACADEMY

Bonus - multi-scale chunking (10 points)

Read this article. Their hypothesis: no single chunk size is optimal for all queries.
Test whether this holds on FinanceBench.

Instructions:

1. Build 2–3 FAISS indices at different chunk sizes. You already have chunk_size=1000 from task 3, so just build 1–2 more (e.g., 300, 2000). Keep the embedding model, chunking method, and overlap policy fixed across indices - chunk size should be the only variable.
2. For each question in the dataset record page-hit@5 for each index.

Discuss:

1. For how many questions does the best-performing chunk size differ?
2. Is there a dominant winner on FinanceBench, or is it query-dependent?

Deliverables:

1. The code
2. A short write-up with a summary table of page-hit@5 per chunk size, the disagreement rate across questions, and a brief discussion of what you found - including whether the article's claim holds in this domain.